

CO-2-AT-03

A.Shiva Shankar Reddy

192425174

COURSE&CODE:DAA&CSA0613

DEBUGGING & OPTIMISATION TASK

Q1. Debugging Selection Sort Code

The following Selection Sort code contains logical errors and produces incorrect output for some inputs. Carefully analyze the code, identify all errors in comparison and swapping logic, and explain their root causes. Apply systematic debugging to correct the algorithm step-by-step. Optimize the implementation by reducing unnecessary comparisons if possible. Analyze the corrected algorithm in terms of time complexity. Rewrite the final optimized code with proper structure and comments.

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if arr[j] > arr[min_idx]: # ERROR  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
    return arr
```

Original code:

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if arr[j] > arr[min_idx]: # ERROR  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
    return arr
```

Step 1 – Identify the Errors

Error 1: Wrong Comparison Operator

Python

```
if arr[j] > arr[min_idx]:
```

Problem

Selection Sort for ascending order should search for the smallest element.

The above code searches for the largest element.

Correct Statement

```
if arr[j] < arr[min_idx]:
```

Error 2: Unnecessary Swap

The code always swaps the elements.

Example:

Minimum element already exists at position i.

Still the algorithm performs

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

which is unnecessary.

Optimization

Swap only

```
min_idx != i
```

Optimized Python Code or Source code

Selection Sort - Debugged and Optimized

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        # Assume first unsorted element is minimum
        min_idx = i
        # Find the smallest element
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        # Swap only if needed
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# Driver Program
numbers = [64, 25, 12, 22, 11]
print("Original Array:")
print(numbers)
sorted_array = selection_sort(numbers)
print("\nSorted Array:")
print(sorted_array)
```

Sample Output

Original Array:

[64, 25, 12, 22, 11]

Sorted Array:

[11, 12, 22, 25, 64]

Output:

main.py	Run	Output
<pre>1 def selection_sort(arr): 2 n = len(arr) 3 4 for i in range(n - 1): 5 min_idx = i 6 for j in range(i + 1, n): 7 8 if arr[j] < arr[min_idx]: 9 min_idx = j 10 11 if min_idx != i: 12 arr[i], arr[min_idx] = arr[min_idx], arr[i] 13 14 return arr 15 numbers = [64, 25, 12, 22, 11] 16 17 print("Original Array:") 18 print(numbers) 19 20 sorted_array = selection_sort(numbers) 21 22 print("\nSorted Array:") 23 print(sorted_array)</pre>		<pre>Original Array: [64, 25, 12, 22, 11] Sorted Array: [11, 12, 22, 25, 64] === Code Execution Successful ===</pre>

Optimization Performed

Before Optimization

Always performs swap

Even when

`min_idx == i`

This wastes execution time.

After Optimization

Swap occurs only when needed.

Advantages

- Fewer write operations
- Slightly faster execution
- Better memory efficiency
- Cleaner implementation

Time Complexity Analysis

Best Case

$O(n^2)$

Even if array is already sorted, Selection Sort still compares every remaining element.

Average Case

$O(n^2)$

Worst Case

$O(n^2)$

Space Complexity

$O(1)$

Only a few extra variables are used.

Comparison Count

Selection Sort performs

$$\frac{n(n-1)}{2}$$

comparisons.

Example

For

$n = 5$

Comparisons

$5 \times 4 / 2 = 10$

Advantages

- Simple algorithm
- Easy to implement
- Requires constant extra memory

Disadvantages

- Slow for large datasets
- Always performs $O(n^2)$ comparisons
- Not adaptive
- Not stable

Conclusion

The original Selection Sort implementation contained two logical issues:

1. It selected the largest element because it used the wrong comparison operator ($>$ instead of $<$).
2. It performed unnecessary swaps even when the minimum element was already in the correct position.

After correcting the comparison logic and swapping only when necessary, the algorithm correctly sorts the array in ascending order while avoiding redundant write operations. Although the optimization reduces unnecessary swaps, the algorithm still has a time complexity of $O(n^2)$ because it must compare each element with the remaining unsorted elements